Atividade prática: investigação de concorrência e sincronização

Disciplina: Sistemas Operacionais
Curso: Graduação em Ciência da Computação
Professor: Frederico Luís Cabral
Modalidade: em dupla
Prazo de entrega: a combinar
Repositório: https://github.com/fredluiscabral/UFCIRJ/tree/main/SO

1. Contexto e objetivos

Em sistemas operacionais, threads podem executar concorrentemente e acessar regiões de memória compartilhadas. Isso permite organizar o trabalho em atividades independentes, mas exige cuidado: operações que parecem simples podem interferir umas nas outras. Neste laboratório, vocês investigarão esses efeitos a partir de códigos fornecidos na disciplina. A proposta é seguir o ciclo **hipótese → execução → registro → análise → nova hipótese**.

Ao final, a dupla deverá conseguir: (i) explicar o funcionamento de um buffer circular compartilhado; (ii) diferenciar problemas de coordenação (buffer cheio/vazio) de condições de corrida; (iii) identificar variáveis compartilhadas e trechos críticos; (iv) analisar `critical`, `atomic` e `reduction` em OpenMP; e (v) interpretar diferenças entre resultados e comportamento de execução sem concluir que a ausência de erro visível implica correção.

Importante: alguns programas são deliberadamente incorretos e possuem *data races*, isto é, acessos conflitantes sem sincronização. Em C, isso pode produzir comportamento indefinido. Uma execução que parece correta não demonstra que o código está correto; também não se deve assumir que um erro específico ocorrerá sempre.

2. Preparação do ambiente

É necessário um ambiente Linux com GCC, Make e suporte a POSIX Threads e OpenMP. No terminal, verifiquem:

```
gcc --version
make --version
```

Clonem o repositório e entrem na pasta da disciplina:

```
git clone https://github.com/fredluiscabral/UFCIRJ.git
cd UFCIRJ/SO
```

Se já houver uma cópia local, acessem-na e atualizem-na com `git pull --ff-only` antes de iniciar. Para abrir a pasta em uma instalação do VS Code que disponibilize o comando `code`, usem `code ..`

Antes de executar os exemplos, observem a estrutura: `exemplos/produtor_consumidor_basico`, `exemplos/produtor_consumidor_diagnostico`, `exemplos/produtor_consumidor_visual` e `exemplos/openmp`. Os slides estão em `slides/`.

Regra do laboratório: registrem as hipóteses *antes* de executar cada experimento. Para alterações de código, anotem exatamente o que mudou; preservem os arquivos originais (usem uma cópia local ou controlem as modificações com Git).

3. Experimento A: produtor-consumidor e buffer circular

A1. Antes de executar: formular hipóteses

Acessem `exemplos/produtor_consumidor_visual/produtor_consumidor.c`, identifiquem `TAM_BUFFER`, `TOTAL_ITENS`, `buffer`, `entrada`, `saida`, as funções `produtor` e `consumidor` e as chamadas `pthread_create`/`pthread_join`.

H1. Um produtor gera os números de 1 a 20 e existe um buffer de cinco posições. O consumidor receberá obrigatoriamente os 20 números, sem perdas ou repetições? Justifiquem.
H2. O que ocorrerá quando o produtor escrever numa posição ainda não consumida?
H3. O que pode acontecer se o consumidor tentar ler uma posição sem item novo?

A2. Executar e observar

```
cd exemplos/produtor_consumidor_visual
make
./produtor_consumidor
```

Observem as linhas `PRODUZ`, `CONSOME` e o conteúdo das cinco posições do buffer. **Nesta versão, o consumidor tem um atraso inicial intencional**, o que facilita visualizar sobrescritas; isso não constitui sincronização. Também há atrasos entre operações. Por isso, a primeira execução pode mostrar muitas produções antes do início do consumo.

Registro A: copiem de 8 a 15 linhas significativas do terminal que incluam uma sobrescrita ou leitura de valor inesperado; expliquem o que mudou no buffer. Não é necessário colar a saída completa.

Questões A:

1. Para que servem `entrada` e `saida`? Por que os índices usam `% TAM_BUFFER`?
2. Identifiquem uma posição que recebeu um novo valor antes que o valor anterior fosse consumido. Quais valores estavam envolvidos?
3. O fato de o consumidor ler repetidamente um valor significa necessariamente que o produtor gerou o mesmo valor mais de uma vez? Expliquem.
4. A saída colorida representa uma ordenação confiável de *todas* as operações de memória? Por quê?
5. Quais trechos do código acessam o mesmo `buffer`? Em que circunstâncias esses acessos podem conflitar?

A3. Modificar uma variável experimental

Façam uma **cópia local** do arquivo e alterem somente o atraso inicial do consumidor, localizado na função `consumidor`:

```
usleep(350000);
```

Troquem por:

```
usleep(0);
```

Antes de recompilar, escrevam **H4:** como essa alteração deverá influenciar a intercalação de produção e consumo? Depois executem `make` e `./produtor_consumidor` novamente. Comparem as primeiras 15 a 20 operações com as da execução original. O resultado pode variar com o escalonamento: descrevam o que realmente observaram.

Questões A3: a mudança resolveu o problema? Mudou apenas a frequência ou a aparência das falhas? Expliquem por que atrasos artificiais não substituem sincronização.

Voltem à pasta `SO` com `cd ../..` antes do próximo experimento.

4. Experimento B: saída com diagnóstico

Agora comparem o exemplo visual com uma variante que exibe um diagnóstico do comportamento.

H5. Se produtor e consumidor executarem o mesmo número de operações, isso basta para garantir que cada item foi entregue corretamente? Justifiquem antes de rodar.

```
cd exemplos/produtor_consumidor_diagnostico
make
./produtor_consumidor
```

Registro B: registrem os dados apresentados no resumo do programa e um trecho representativo. Se houver divergências, expliquem o que elas indicam. Se não houver, expliquem por que uma execução isolada não demonstra ausência de problemas.

Questões B:

1. Qual diferença entre “20 tentativas de consumo” e “20 itens distintos corretamente consumidos”?
2. O diagnóstico altera a correção do acesso concorrente ao buffer ou apenas ajuda a observar o comportamento?
3. Quais condições precisariam ser garantidas para um buffer limitado funcionar corretamente? Separem as exigências de **exclusão mútua** e **coordenação**.

Retornem à pasta `SO` com `cd ../...`

5. Experimento C: OpenMP, threads e condição de corrida

C1. Primeira execução paralela

```
cd exemplos/openmp
make
./01_threads
```

H6. A ordem de impressão dos identificadores das threads será sempre a mesma? Por quê?

Registro C1: anotem a saída de duas execuções e indiquem o que, se algo, mudou.

C2. Contador sem sincronização

Antes de executar, abram `02_corrida.c` e identifiquem a variável compartilhada `contador`, o número de threads solicitado e a diretiva `parallel for`.

H7. Qual seria o valor esperado após 1.000.000 de incrementos? Por que o programa pode não produzir esse valor?

```
./02_corrida
./02_corrida
./02_corrida
./02_corrida
./02_corrida
```

Registro C2: preencham uma tabela com cinco execuções: número da execução, valor esperado, valor obtido e observações. Não é necessário que os valores sejam diferentes entre si.

Questões C:

1. `contador++` corresponde necessariamente a uma única operação atômica? Representem conceitualmente suas etapas.
2. Construam uma intercalação de duas threads que partem do mesmo valor e provoque uma *atualização perdida*.
3. O que significa dizer que esse código apresenta *data race* e comportamento indefinido em C?
4. Qual diferença entre o problema do buffer que sobrescreve dados e o problema da atualização do contador?

6. Experimento D: diferentes formas de tratar o contador

Antes de executar os próximos programas, leiam `03_critical.c`, `04_atomic.c` e `05_reduction.c` e preencham as hipóteses:

H8. Qual mecanismo impede que duas threads executem a mesma região `critical` simultaneamente?
H9. Em que sentido `atomic` é mais restrita que `critical`?
H10. Por que `reduction` não é apenas outra forma de proteger cada incremento com um lock?

Executem:

```
./03_critical
./04_atomic
./05_reduction
```

Registro D: montem uma tabela comparando os quatro programas `02_corrida`, `03_critical`, `04_atomic` e `05_reduction` com as colunas: mecanismo empregado; valor observado; motivo pelo qual o resultado é ou não garantido; e forma como o contador é acessado.

Em seguida, façam um **experimento de variação**: trabalhem em cópias dos arquivos (não alterem os originais), mudem o número de threads solicitado de quatro para dois e, depois, para oito. Recompilem a cada alteração. Anotem quais diferenças foram observadas. Se o computador disponibilizar menos núcleos, considerem esse fato na análise.

Questões D:

1. `critical` garante a ordem em que as threads entram na região crítica? Justifiquem.
2. `atomic` pode proteger qualquer sequência de comandos em C? Expliquem com base no código.
3. Como `reduction(+:contador)` organiza os valores usados pelas threads durante a execução e ao final?
4. Resultados numericamente iguais significam que os três mecanismos têm o mesmo funcionamento interno? Por quê?
5. Se uma solução garante o resultado correto, isso demonstra automaticamente que ela é a mais rápida? Qual experimento adicional seria necessário para discutir desempenho?

Desafio opcional: comparem os tempos de execução das versões em uma máquina disponível (por exemplo, com `/usr/bin/time -p ./03_critical`), repetindo as medições e registrando as condições experimentais. Não tratem uma única medição como conclusão definitiva. A versão sem sincronização não deve ser classificada como “mais eficiente” apenas porque executa menos trabalho de sincronização: ela não oferece o mesmo requisito de correção.

7. Síntese conceitual

Respondam, com suas próprias palavras:

1. Definam **região crítica**, **exclusão mútua** e **condição de corrida**. Relacionem cada conceito a uma linha de um dos códigos estudados.
2. Por que a proteção de uma região crítica não resolve, sozinha, as situações de buffer cheio e buffer vazio?
3. Qual relação entre o algoritmo de Peterson discutido em aula e a diretiva `critical` do OpenMP? Apontem uma semelhança no objetivo e uma diferença na forma de utilização.
4. Proponham uma mudança simples em algum código estudado e apresentem uma nova hipótese testável. Não é necessário implementar uma solução completa de sincronização.

8. Entrega

Em dupla, com data a combinar. Entreguem um relatório em PDF, organizado pelos quatro experimentos, contendo:

- identificação dos dois integrantes;
- hipóteses formuladas **antes** dos testes;
- comandos executados e alterações realizadas nos parâmetros;
- tabelas e **trechos selecionados** das saídas (capturas de tela são opcionais);
- respostas fundamentadas às questões;
- comparação final entre as hipóteses e os resultados, incluindo pelo menos um caso em que a observação divergiu do esperado ou uma explicação de por que isso não ocorreu.

Não é necessário copiar o código integral para o relatório. Sempre que mencionarem uma implementação, indiquem o arquivo e o trecho relevante. Não basta responder apenas “funcionou” ou “deu erro”: descrevam a evidência observada e o mecanismo envolvido.

Leitura de apoio

Para fundamentar a análise, consultem os **livros oficiais da disciplina**, especialmente os capítulos sobre processos, threads, concorrência, regiões críticas e sincronização. Consultem também o material de aula em `SO/slides/`. Usem as leituras para **explicar os resultados dos experimentos**, relacionando conceitos e observações, em vez de reproduzir definições isoladas.